

Explanation

Bcrypt

- **Bcrypt ek password hashing library hai** jo ki passwords ko **secure** banane ke liye use hoti hai.
- Yeh hashing algorithm **one-way encryption** karta hai, iska matlab ek baar password encrypt hone ke baad use decrypt nahi kiya ja sakta.

JWT (JSON Web Token)

- JWT ek **token-based authentication** mechanism hai jo **stateless** hota hai.
- Matlab **server ko session maintain nahi karna padta**
- Ek baar token generate ho gaya, toh **user har request me usye bhej sakta hai** taaki authentication ho sake

.

Pehle Samjhein: Session Kya Hota Hai?

Session-Based Authentication (Traditional Approach)

Jab ek user **login** karta hai, toh **server ek session create karta hai aur usme user ki details store karta hai**.

Phir **client ko ek session ID di jati hai**, jo har request ke sath bhejni padti hai.

◆ Flow:

1. **User login karta hai** → Server ek **session create karta hai**.
2. Server **session ID client ko bhejta hai** (cookies me ya headers me store hota hai).
3. **Client jab bhi request bhejta hai**, toh session ID bhejni padti hai.
4. Server **session ID check karta hai** aur agar valid hoti hai toh user ko access milta hai.

🔴 Problem:

- Har user ka **session server memory me store hota hai**. Agar bohot sare users aa jayein, toh **server overload ho sakta hai**.
- **Scalability issue hota hai**, kyunki agar multiple servers ho toh sab servers ko session ka pata hona chahiye.
- **Session hijacking** ka risk hota hai (agar session ID leak ho gayi toh dusra user login ho sakta hai).

JWT Stateless Authentication (Modern Approach)

JWT Stateless Kyu Hai?

JWT me **server koi session store nahi karta**. Isme **authentication data token ke andar hota hai**, jo **client ke paas hi rehta hai**.

◆ Flow:

1. **User login karta hai** → Server ek **JWT token generate karta hai**.
2. **Token client ke paas store hota hai** (localStorage ya cookies me).
3. **Jab client request bhejta hai**, toh **token headers me bhejta hai**.
4. Server **JWT verify karta hai** aur agar valid hota hai toh access deta hai.

✅ Advantages:

- **No Session Storage on Server** → Server ko **session maintain nahi karna padta**.
- **Highly Scalable** → Kyunki **server stateless hota hai**, toh multiple servers ke sath easily kaam kar sakta hai.
- **More Secure** → Session hijacking ka risk kam hota hai, kyunki token **signed aur encrypted hota hai**.

bcrypt.genSaltSync(10) & bcrypt.hashSync(password, salt) ka Detailed Explanation

Yeh **do methods** password **hashing aur security** ke liye use hote hain.

- **bcrypt.genSaltSync(10)** → Ek **salt generate karta hai**.

- `bcrypt.hashSync(password, salt)` → Us salt ka use karke **password hash** karta hai.

Step 1: Salt Generate Karna (`bcrypt.genSaltSync(10)`)

```
const salt = bcrypt.genSaltSync(10);
```

Salt kya hota hai?

- **Salt ek random string hoti hai** jo password hashing ke saath add hoti hai.
- Iska **motive** yeh hai ki **same password different hashes produce kare**.
- Salt add hone se **password guessing (rainbow table attack)** mushkil ho jata hai.

10 ka matlab kya hai?

- **10** yahan **rounds of salting** ko represent karta hai.
- Matlab hashing **10 baar process hogi** taaki strong security mile.
- Higher number = **More security but slow processing**.
- Generally, **10 se 12 rounds recommended** hote hain.

◆ **Example (Salt Generate hoke kuch aisa dikhega):**

```
$2b$10$XnPiYOyW3hNdHp9yQdxJvO
```

◆ **Step 2: Password Hash Karna** (`bcrypt.hashSync(password, salt)`)

```
const hashedPassword = bcrypt.hashSync(password, salt);
```

Yeh kya karta hai?

- User ka original password leta hai.
- Salt ke saath mix karta hai.
- Hashed password generate karta hai jo irreversible hota hai (isko decode nahi kiya ja sakta).

◆ Example:

Agar user ka password "mypassword" hai, toh hashing ke baad kuch aisa dikhega:

```
$2b$10$XnPiYOyW3hNdHp9yQdxJvO5Z7S9P6C0e/fJY/Jj6zLv3e2bPQoZXe
```

🛑 Notice:

- "mypassword" ka hashed version totally alag hai.
- Even agar do users ka password same ho, toh unke hashes different honge (kyunki salt har baar different hoti hai).